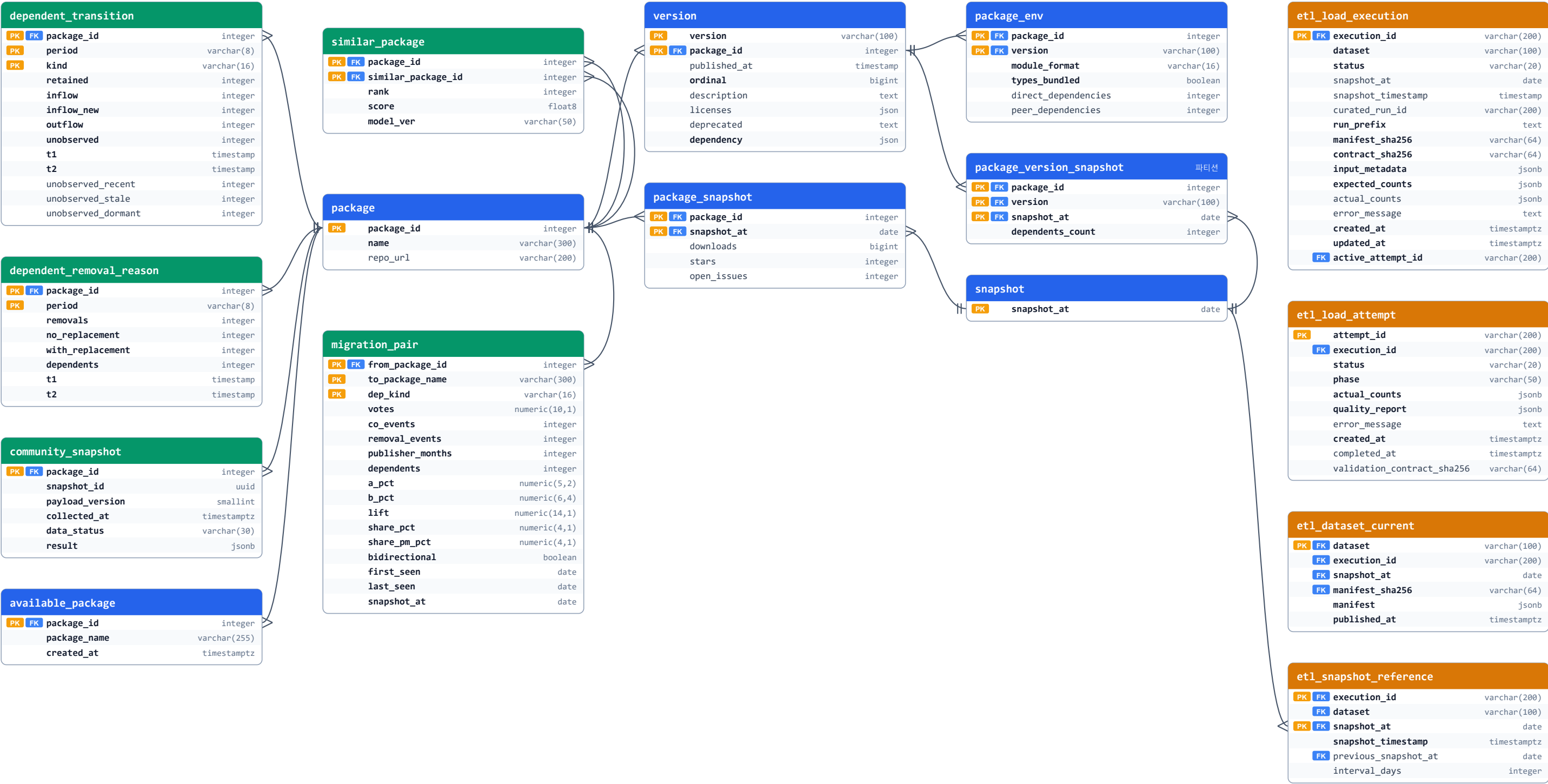


Pickage ERD

PostgreSQL 16 · Flyway V1~V13 적용 스키마 · 테이블 16개 · 컬럼 117개

기준 커밋 bda8f78 · 생성 2026-09-24
굵은 컬럼 = NOT NULL

패키지-스냅샷 (서비스 핵심) 분석 결과 (배치 계시) ETL 적재 이력 (데이터 계시 관리) 기본 키 외래 키 N : 1 (까마귀발 = 참조하는 쪽 여러 행)



1. 개요

Pickage 서비스 DB 는 세 영역으로 나뉜다. **패키지·스냅샷**은 deps.dev-npm 에서 수집한 원천 사실을 스냅샷 기준일 축으로 저장하고, **분석 결과**는 Spark-Python 배치와 AI 유사도 배치가 계산해 게시한 결과를 API 가 키 조회로 읽는다. **ETL 적재 이력**은 어떤 입력을 언제 게시했는지를 기록해 게시를 원자적으로 교체하고 재현할 수 있게 한다.

스키마는 백엔드의 Flyway 마이그레이션(backend/src/main/resources/db/migration)이 소유하며, 이 문서는 그 마이그레이션을 빈 DB 에 적용한 결과에서 자동 생성했다.

영역	테이블	설명	컬럼
● 패키지·스냅샷	available_package	서비스에서 검색·분석 가능한 패키지 목록(데이터가 갖춰진 패키지만)	3
● 패키지·스냅샷	package	npm 패키지 마스터. deps.dev 전수 적재(약 1,100만 행)	3
● 패키지·스냅샷	version	패키지 버전별 메타데이터(배포 시각·설명·라이선스·폐가·의존 선언)	8
● 패키지·스냅샷	package_snapshot	기준일별 패키지 지표 — 주간 다운로드·GitHub stars·열린 이슈	5
● 패키지·스냅샷	package_env	버전별 소비 조건 — 모듈 방식·타입 동봉·직접/peer 의존 수 (기능 비교)	6
● 패키지·스냅샷	package_version_snapshot	기준일·버전별 의존 수. snapshot_at 범위 파티션 테이블	4
● 패키지·스냅샷	snapshot	deps.dev 스냅샷 기준일 목록. 모든 시계열 지표의 날짜 축	1
● 분석 결과	dependent_transition	구간(1y·3y·5y) 양 끝 의존자 집합 비교 — 유지·유입·이탈	13
● 분석 결과	dependent_removal_reason	구간별 이탈 사유 — 대체 동반 / 대체 없음	8
● 분석 결과	community_snapshot	GitHub 커뮤니티 분석 결과(패키지당 최신 1건, 7일 제공)	6
● 분석 결과	similar_package	AI 유사도 배치가 계산한 대체 후보 순위 (유사 후보 추천)	5
● 분석 결과	migration_pair	관측된 교체 흐름 — X 를 빼고 Y 를 넣은 이동 쌍과 통계	17
● ETL 적재 이력	etl_load_execution	데이터셋 적재 실행 단위. 입력 매니페스트·계약 해시·게시 상태	16
● ETL 적재 이력	etl_load_attempt	적재 실행의 개별 시도와 단계·품질 보고	10
● ETL 적재 이력	etl_dataset_current	데이터셋별로 현재 서비스에 게시된 실행(원자적 포인터)	6
● ETL 적재 이력	etl_snapshot_reference	실행별 스냅샷 날짜 계보(원천 시각·직전 기준일·간격)	6

관계 요약

참조하는 테이블 (N)	컬럼	참조되는 테이블 (1)	비고
available_package	package_id	package (package_id)	삭제 연쇄
version	package_id	package (package_id)	
package_snapshot	package_id	package (package_id)	
package_snapshot	snapshot_at	snapshot (snapshot_at)	
package_env	package_id, version	version (package_id, version)	
package_version_snapshot	package_id, version	version (package_id, version)	
package_version_snapshot	snapshot_at	snapshot (snapshot_at)	
dependent_transition	package_id	package (package_id)	
dependent_removal_reason	package_id	package (package_id)	
community_snapshot	package_id	package (package_id)	삭제 연쇄
similar_package	package_id	package (package_id)	
similar_package	similar_package_id	package (package_id)	
migration_pair	from_package_id	package (package_id)	
etl_load_execution	execution_id, active_attempt_id	etl_load_attempt (execution_id, attempt_id)	지연 검사
etl_load_attempt	execution_id	etl_load_execution (execution_id)	
etl_dataset_current	dataset, execution_id, snapshot_at, manifest_sha256	etl_load_execution (dataset, execution_id, snapshot_at, manifest_sha256)	
etl_snapshot_reference	snapshot_at	snapshot (snapshot_at)	
etl_snapshot_reference	dataset, execution_id	etl_load_execution (dataset, execution_id)	
etl_snapshot_reference	execution_id, previous_snapshot_at	etl_snapshot_reference (execution_id, snapshot_at)	자기 참조

2. 테이블 정의서

패키지-스냅샷 2.1 available_package

서비스에서 검색·분석 가능한 패키지 목록(데이터가 갖춰진 패키지만)

#	컬럼	타입	NULL	기본값	키	설명
1	package_id	integer	N		PK FK	패키지 식별자 → package.package_id
2	package_name	varchar(255)	N			패키지명 (검색용 비정규화 사본)
3	created_at	timestampz	N	CURRENT_TIMESTAMP		목록 등록 시각

패키지-스냅샷 2.2 package

npm 패키지 마스터. deps.dev 전수 적재(약 1,100만 행)

#	컬럼	타입	NULL	기본값	키	설명
1	package_id	integer	N		PK	패키지 식별자 (deps.dev 기준 정수 ID)
2	name	varchar(300)	N			npm 패키지명. 스코프 포함(@scope/name), 유일
3	repo_url	varchar(200)	Y			소스 저장소 주소(GitHub 등). 없을 수 있음

- UNIQUE (name)
- INDEX idx_package_name_prefix btree (name text_pattern_ops)

패키지-스냅샷 2.3 version

패키지 버전별 메타데이터(배포 시각·설명·라이선스·패키지 의존 선언)

#	컬럼	타입	NULL	기본값	키	설명
1	version	varchar(100)	N		PK	버전 문자열 (semver)
2	package_id	integer	N		PK FK	패키지 식별자 → package.package_id
3	published_at	timestamp	Y			npm 배포 시각
4	ordinal	bigint	N	0		버전 정렬 순번. 문자열 정렬 오류(4.9.0 > 4.19.2)를 피하려고 최신 버전 판정에 쓴다
5	description	text	Y			해당 버전의 package.json description
6	licenses	json	Y			라이선스 목록 (JSON 배열)
7	deprecated	text	Y			npm deprecate 메시지. NULL 이면 폐기되지 않음
8	dependency	json	N	{ "dependencies": {}, ...		유저에게 의존성 보여주는 용도, 따로 계산할때 쓰진 않음

- INDEX idx_version_pkg_ordinal btree (package_id, ordinal DESC)

패키지-스냅샷 2.4 package_snapshot

기준일별 패키지 지표 — 주간 다운로드·GitHub stars·열린 이슈

#	컬럼	타입	NULL	기본값	키	설명
1	package_id	integer	N		PK FK	패키지 식별자 → package.package_id
2	snapshot_at	date	N		PK FK	스냅샷 기준일 → snapshot.snapshot_at
3	downloads	bigint	Y			기준일 직전 구간의 주간 다운로드 수
4	stars	integer	Y			관측 시점의 GitHub stars
5	open_issues	integer	Y			관측 시점의 열린 이슈 수

패키지-스냅샷 2.5 package_env

버전별 소비 조건 — 모듈 방식·타입·의존 조건 (기능-11 첫 결과 카드)

#	컬럼	타입	NULL	기본값	키	설명
1	package_id	integer	N		PK FK	패키지 식별자 → version.package_id
2	version	varchar(100)	N		PK FK	버전 → version.version
3	module_format	varchar(16)	N			어떻게 불러오는가. CJS · ESM_ONLY · ESM_CJS(듀얼) · UNKNOWN(unpublish)

#	컬럼	타입	NULL	기본값	키	설명
4	types_bundled	boolean	N			타입 선언이 패키지에 동봉됐는가. 거짓은 @types 별도 설치를 뜻하며 타입 없음이 아니다
5	direct_dependencies	integer	Y			dependencies 선언 수. 전이 포함 아님. NULL 은 unpublish 라 모름이며 0 이 아니다
6	peer_dependencies	integer	Y			peerDependencies 선언 수. 사용자가 직접 맞춰야 하는 조건. NULL 은 모름

- CHECK (((module_format)::text = ANY ((ARRAY['CJS'::character varying, 'ESM_ONLY'::character varying, 'ESM_CJS'::character varying, 'UNKNOWN'::character varying]))::text[])))
- CHECK (((direct_dependencies IS NULL) OR (direct_dependencies >= 0)) AND ((peer_dependencies IS NULL) OR (peer_dependencies >= 0))))

패키지-스냅샷 2.6 package_version_snapshot

기준일-버전별 의존 수. snapshot_at 범위 파티션 테이블

파티션 테이블 — PARTITION BY RANGE (snapshot_at). 스냅샷 기준일 범위마다 파티션을 적재 시점에 만든다.

#	컬럼	타입	NULL	기본값	키	설명
1	package_id	integer	N		PK FK	패키지 식별자 → version.package_id
2	version	varchar(100)	N		PK FK	버전 → version.version
3	snapshot_at	date	N		PK FK	스냅샷 기준일 (파티션 키) → snapshot.snapshot_at
4	dependents_count	integer	N	0		그 기준일에 이 버전을 의존하는 패키지 수

- CHECK ((dependents_count >= 0))
- INDEX idx_pvs_pkg_snapshot btree (package_id, snapshot_at)

패키지-스냅샷 2.7 snapshot

deps.dev 스냅샷 기준일 목록. 모든 시계열 지표의 날짜 축

#	컬럼	타입	NULL	기본값	키	설명
1	snapshot_at	date	N	'2026-08-31'::date	PK	BigQuery의 snapshot 테이블 값을 기준으로 함

분석 결과 2.8 dependent_transition

구간 양 끝의 dependent 선언 집합 비교 (기능-08)

#	컬럼	타입	NULL	기본값	키	설명
1	package_id	integer	N		PK FK	패키지 식별자 → package.package_id
2	period	varchar(8)	N		PK	구간 프리셋. 1y · 3y · 5y
3	kind	varchar(16)	N		PK	선언 종류. regular · peer · optional
4	retained	integer	N			T1-T2 모두 의존하는 의존자 수 (유지)
5	inflow	integer	N			T1 에는 없고 T2 에 새로 의존하는 의존자 수 (유입)
6	inflow_new	integer	N			유입 중 T1 때 아직 없던 패키지. inflow 의 부분집합이며 채택이 아니라 생태계 성장
7	outflow	integer	N			T1 에는 의존했으나 T2 에 뺀 의존자 수 (이탈)
8	unobserved	integer	N			구간 안에 대표 릴리스가 안 바뀌어 판정할 수 없는 dependent. 유지로 세면 안 된다
9	t1	timestamp	N			구간 시작. period 로 정해지는 상수
10	t2	timestamp	N			구간 끝 = 원천 스냅샷 날짜. 오늘이 아니다
11	unobserved_recent	integer	Y			관측불가 중 마지막 대표 릴리스가 t2 기준 3년 안. 3y·5y 구간에서는 정의상 0
12	unobserved_stale	integer	Y			관측불가 중 마지막 대표 릴리스가 t2 기준 3~5년 전. 5y 구간에서는 정의상 0
13	unobserved_dormant	integer	Y			관측불가 중 마지막 대표 릴리스가 t2 기준 5년 초과. 사실상 방치된 프로젝트다

- CHECK (((period)::text = ANY ((ARRAY['1y'::character varying, '3y'::character varying, '5y'::character varying]))::text[])))
- CHECK (((kind)::text = ANY ((ARRAY['regular'::character varying, 'peer'::character varying, 'optional'::character varying]))::text[])))
- CHECK (((retained >= 0) AND (inflow >= 0) AND (inflow_new >= 0) AND (outflow >= 0) AND (unobserved >= 0)))
- CHECK ((inflow_new <= inflow))
- CHECK ((t1 < t2))
- CHECK (((unobserved_recent IS NULL) AND (unobserved_stale IS NULL) AND (unobserved_dormant IS NULL)) OR ((unobserved_recent >= 0) AND (unobserved_stale >= 0) AND (unobserved_dormant >= 0) AND (((unobserved_recent + unobserved_stale) + unobserved_dormant) = unobserved))))
- CHECK (((unobserved_recent IS NULL) OR ((period)::text = '1y'::text) OR (((period)::text = '3y'::text) AND (unobserved_recent = 0)) OR (((period)::text = '5y'::text) AND (unobserved_recent = 0) AND (unobserved_stale = 0))))

분석 결과 2.9 dependent_removal_reason

구간별 이탈 사유 — 대체 동반/대체 없음 (기능-08). 단위는 전이 건수다

#	컬럼	타입	NULL	기본값	키	설명
1	package_id	integer	N		PK FK	패키지 식별자 → package.package_id
2	period	varchar(8)	N		PK	구간 프리셋. 1y · 3y · 5y. dependent_transition 과 같은 값이며 같은 t2 에서 끝난다
3	removals	integer	N			X 를 뺀 전이의 수. 패키지 수가 아니다 — 한 의존자가 여러 번 빼면 여러 건이다
4	no_replacement	integer	N			뺀 릴리스에서 아무것도 새로 넣지 않은 전이
5	with_replacement	integer	N			뺀 릴리스에서 다른 것을 함께 넣은 전이. 같은 자리의 대체라는 보장은 없다
6	dependents	integer	N			X 를 뺀 적 있는 의존자 수(중복 접음). removals 와 단위가 다르므로 나누지 말 것
7	t1	timestamp	N			구간 시작. period 로 정해지는 상수
8	t2	timestamp	N			구간 끝 = 원천 스냅샷 날짜. 오늘이 아니다

- CHECK (((period)::text = ANY ((ARRAY['1y'::character varying, '3y'::character varying, '5y'::character varying]))::text[])))
- CHECK (((removals > 0) AND (no_replacement >= 0) AND (with_replacement >= 0) AND (dependents > 0)))
- CHECK (((no_replacement + with_replacement) = removals))
- CHECK ((dependents <= removals))
- CHECK ((t1 < t2))

분석 결과 2.10 community_snapshot

GitHub 커뮤니티 분석 결과(패키지당 최신 1건, 7일 제공)

#	컬럼	타입	NULL	기본값	키	설명
1	package_id	integer	N		PK FK	패키지당 최신 결과 한 행만 보관한다(PK). package 삭제 시 함께 지운다 → package.package_id
2	snapshot_id	uuid	N			응답 버전 식별과 동시 갱신 결과 구분용. 원문-source ID는 여기 넣지 않는다
3	payload_version	smallint	N			배포 후 오래된 result JSON 형식을 안전하게 판별한다. 1부터 시작
4	collected_at	timestampz	N			24시간 재사용-7일 제공-메시간 정리 배치 판정의 유일한 기준 시작
5	data_status	varchar(30)	N			AVAILABLE-PARTIAL-UNVERIFIED_REPOSITORY-AMBIGUOUS_SCOPE-UNSUPPORTED_HOST-NO_DISCUSSION_DATA 중 하나. 일시적 외부 호출 실패-진행 중 상태는 이 테이블에 저장하지 않는다
6	result	jsonb	N			검증된 저장소 식별자-이슈 최대 2개-논의 흐름-대표 메시지-제한 사유. 원문 본문-댓글-prompt-hash는 넣지 않는다

- UNIQUE (snapshot_id)
- CHECK ((payload_version > 0))
- CHECK (((data_status)::text = ANY ((ARRAY['AVAILABLE'::character varying, 'PARTIAL'::character varying, 'UNVERIFIED_REPOSITORY'::character varying, 'AMBIGUOUS_SCOPE'::character varying, 'UNSUPPORTED_HOST'::character varying, 'NO_DISCUSSION_DATA'::character varying]))::text[])))
- CHECK ((jsonb_typeof(result) = 'object'::text))
- INDEX idx_community_snapshot_collected_at btree (collected_at)

분석 결과 2.11 similar_package

AI 유사도 배치가 계산한 대체 후보 순위 (유사 후보 추천)

#	컬럼	타입	NULL	기본값	키	설명
1	package_id	integer	N		PK FK	기존 패키지 → package.package_id
2	similar_package_id	integer	N		PK FK	대체 후보 패키지 → package.package_id
3	rank	integer	N			순위
4	score	float8	N			종합 점수(유사도+기타)
5	model_ver	varchar(50)	N			판정 모델 버전

- UNIQUE (package_id, rank)
- CHECK ((package_id <> similar_package_id))
- CHECK (((rank >= 1) AND (rank <= 50)))

분석 결과 2.12 migration_pair

관측된 교체 흐름 — X 를 빼고 Y 를 넣은 쌍 (확장-02). 단위는 쌍이다

#	컬럼	타입	NULL	기본값	키	설명
1	from_package_id	integer	N		PK FK	출발 패키지(뺀 쪽) → package.package_id

#	컬럼	타입	NULL	기본값	키	설명
2	to_package_name	varchar(300)	N		PK	도착 패키지 이름. package 에 없을 수 있다 — FK 가 아닌 이유는 파일 머리말 참고
3	dep_kind	varchar(16)	N		PK	의존 종류이자 원천. regular=deps.dev 전수 · dev=registry 상위 10만. 모집단이 달라 합치지 않는다
4	votes	numeric(10,1)	N			한 전이에서 함께 들어온 후보가 k 개면 1/k 씩 나눠 준 표의 합. 건수가 아니다
5	co_events	integer	N			X 를 빼면서 Y 를 넣은 전이 수. votes 와 달리 나누지 않은 건수
6	removal_events	integer	N			X 를 뺀 전이 수 전체. 이 쌍의 분모다
7	publisher_months	integer	N			서로 다른 (배포주체, 달) 조합 수. 한 조직의 일괄 변경을 걸러 내는 핵심 값이다
8	dependents	integer	N			이 이동을 한 의존자 수(중복 접음)
9	a_pct	numeric(5,2)	N			co_events / removal_events × 100. X 를 뺀 사람 중 Y 를 넣은 비율
10	b_pct	numeric(6,4)	N			모집단에서 Y 가 추가되는 기준율 × 100. lift 의 분모
11	lift	numeric(14,1)	N			a_pct / b_pct. 우연 대비 몇 배인가. 모집단이 다르면 절댓값을 비교할 수 없다
12	share_pct	numeric(4,1)	N			출발 패키지의 표 합 중 이 도착치 비율. 라벨 필터용이며 화면에는 쓰지 않는다
13	share_pm_pct	numeric(4,1)	N			조직·달 수 기준 점유율. 화면 점유율은 이것을 쓴다(S15P21A506-136 3번)
14	bidirectional	boolean	N			(Y, X) 쌍도 loose 를 통과한다. 같은 물건의 두 포장(lodash↔lodash-es)일 수 있어 지우지 않고 변종으로 표시한다
15	first_seen	date	N			이 이동이 처음 관측된 날
16	last_seen	date	N			마지막으로 관측된 날
17	snapshot_at	date	N			이 회차가 어디까지 본 원천인지. dep_kind 로 정해지는 상수이며 두 원천이 서로 다르다

- CHECK (((votes <= (co_events)::numeric) AND (co_events <= removal_events)))
- CHECK (((dep_kind)::text = ANY ((ARRAY['regular'::character varying, 'dev'::character varying]))::text[])))
- CHECK (((to_package_name)::text <> ''::text))
- CHECK (((lift >= (5)::numeric) AND (votes >= (3)::numeric)))
- CHECK (((co_events > 0) AND (removal_events > 0) AND (publisher_months > 0) AND (dependents > 0)))
- CHECK (((a_pct >= (0)::numeric) AND (a_pct <= (100)::numeric) AND (b_pct >= (0)::numeric) AND (b_pct <= (100)::numeric) AND (share_pct >= (0)::numeric) AND (share_pct <= (100)::numeric) AND (share_pm_pct >= (0)::numeric) AND (share_pm_pct <= (100)::numeric)))
- CHECK ((first_seen <= last_seen))

ETL 적재 이력 2.13 etl_load_execution

데이터셋 적재 실행 단위. 입력 매니페스트·계약 해시·게시 상태

#	컬럼	타입	NULL	기본값	키	설명
1	execution_id	varchar(200)	N		PK FK	실행 식별자 → etl_load_attempt.execution_id
2	dataset	varchar(100)	N			데이터셋 이름 (package-version, package-snapshot 등)
3	status	varchar(20)	N			PREPARING · FAILED · PUBLISHED
4	snapshot_at	date	Y			입력의 기준일. snapshot-reference 데이터셋은 NULL
5	snapshot_timestamp	timestamp	Y			Curated report 공급자 관측 시각: 원천의 timezone 없는 마이크로초 값 보존
6	curated_run_id	varchar(200)	Y			MinIO Curated 실행 ID
7	run_prefix	text	N			MinIO 입력 경로 접두사
8	manifest_sha256	varchar(64)	N			입력 매니페스트 SHA-256
9	contract_sha256	varchar(64)	N			최초 게시 시 검증한 코드·스키마 계약 해시
10	input_metadata	jsonb	N			입력 메타데이터
11	expected_counts	jsonb	N			매니페스트가 선언한 기대 행 수
12	actual_counts	jsonb	Y			실제 적재 행 수
13	error_message	text	Y			실패 사유
14	created_at	timestampz	N	clock_timestamp()		생성 시각
15	updated_at	timestampz	N	clock_timestamp()		갱신 시각
16	active_attempt_id	varchar(200)	N		FK	현재 유효한 시도 (etl_load_attempt 참조, 지연 검사 FK) → etl_load_attempt.attempt_id

- CHECK (((status)::text = ANY ((ARRAY['PREPARING'::character varying, 'FAILED'::character varying, 'PUBLISHED'::character varying]))::text[])))
- CHECK (((manifest_sha256)::text ~ '^'[0-9a-f]{64}\$'::text))
- CHECK (((contract_sha256)::text ~ '^'[0-9a-f]{64}\$'::text))
- UNIQUE (dataset, execution_id, snapshot_at, manifest_sha256)
- UNIQUE (dataset, execution_id)

- CHECK (((((dataset)::text = 'snapshot-reference'::text) AND (snapshot_at IS NULL) AND (snapshot_timestamp IS NULL) AND (curated_run_id IS NULL)) OR (((dataset)::text <> 'snapshot-reference'::text) AND (snapshot_at IS NOT NULL) AND (snapshot_timestamp IS NOT NULL) AND (curated_run_id IS NOT NULL))))
- INDEX ix_etl_load_execution_input btree (dataset, manifest_sha256, status)

ETL 적재 이력 2.14 etl_load_attempt

적재 실행의 개별 시도와 단계-품질 보고

#	컬럼	타입	NULL	기본값	키	설명
1	attempt_id	varchar(200)	N		PK	시도 식별자
2	execution_id	varchar(200)	N		FK	소속 실행 → etl_load_execution.execution_id
3	status	varchar(20)	N			PREPARING · FAILED · PUBLISHED · REVERIFIED
4	phase	varchar(50)	N			마지막으로 도달한 적재 단계
5	actual_counts	jsonb	N	'{}'::jsonb		이 시도에서 실제로 적재한 행 수
6	quality_report	jsonb	N	'{}'::jsonb		품질 검증 결과
7	error_message	text	Y			실패 사유
8	created_at	timestampz	N	clock_timestamp()		시도 시작 시각
9	completed_at	timestampz	Y			시도 종료 시각. PREPARING 이면 NULL
10	validation_contract_sha256	varchar(64)	Y			이번 시도에서 검증한 코드-스키마 계약. 최초 게시 execution.contract_sha256은 보존

- CHECK (((status)::text = ANY ((ARRAY['PREPARING'::character varying, 'FAILED'::character varying, 'PUBLISHED'::character varying, 'REVERIFIED'::character varying]))::text[])))
- CHECK (((((status)::text = 'PREPARING'::text) AND (completed_at IS NULL)) OR (((status)::text <> 'PREPARING'::text) AND (completed_at IS NOT NULL))))
- UNIQUE (execution_id, attempt_id)
- CHECK (((validation_contract_sha256)::text ~ '^[0-9a-f]{64}\$'::text))
- INDEX ix_etl_load_attempt_execution btree (execution_id, created_at)

ETL 적재 이력 2.15 etl_dataset_current

DB에 원자적으로 게시한 dataset별 현재 입력. MinIO _current.json과 별도

#	컬럼	타입	NULL	기본값	키	설명
1	dataset	varchar(100)	N		PK FK	데이터셋 이름 → etl_load_execution.dataset
2	execution_id	varchar(200)	N		FK	현재 게시된 실행 → etl_load_execution.execution_id
3	snapshot_at	date	N		FK	게시된 실행의 기준일 → etl_load_execution.snapshot_at
4	manifest_sha256	varchar(64)	N		FK	게시된 입력 매니페스트 해시 → etl_load_execution.manifest_sha256
5	manifest	jsonb	N			게시된 입력 매니페스트 원문
6	published_at	timestampz	N	clock_timestamp()		게시 시각

ETL 적재 이력 2.16 etl_snapshot_reference

Projects 기준 목록의 실행별 날짜/원천 시각/직전 날짜 계보. 지표 준비 원료를 의미하지 않음

#	컬럼	타입	NULL	기본값	키	설명
1	execution_id	varchar(200)	N		PK FK	소속 실행 → etl_load_execution.execution_id, etl_snapshot_reference.execution_id
2	dataset	varchar(100)	N	'snapshot-reference'::character varying	FK	항상 'snapshot-reference' → etl_load_execution.dataset
3	snapshot_at	date	N		PK FK	관측된 스냅샷 기준일 → snapshot.snapshot_at
4	snapshot_timestamp	timestampz	N			원천(deps.dev Projects) 관측 시각
5	previous_snapshot_at	date	Y		FK	직전 기준일 (같은 실행 안) → etl_snapshot_reference.snapshot_at
6	interval_days	integer	Y	(snapshot_at - previous_snapshot_at)		직전 기준일과의 간격(일). 생성 컬럼

- CHECK (((dataset)::text = 'snapshot-reference'::text))

- **CHECK** (((snapshot_timestamp AT TIME ZONE 'UTC')::text)::date = snapshot_at))
- **CHECK** (((previous_snapshot_at IS NULL) OR (previous_snapshot_at < snapshot_at)))
- **INDEX** ix_etl_snapshot_reference_date btree (snapshot_at)